3 Test Data Preparation with Selenium

Introduction

Test Data refers to the specific set of input values that are given to a software application during testing to check whether it produces the expected output. Good test data is very important because the accuracy of the testing process directly depends on the quality and variety of data used.

In manual testing, test data is entered directly while performing test steps. However, in Selenium automation, it is a good practice to keep test data separate from the test script. This approach is called Data-Driven Testing. Instead of hard-coding values like username and password inside the script, the data is stored externally in an Excel sheet or CSV file, and the Selenium script reads this data at run time and executes the same test case repeatedly with different sets of values.

Data-Driven Testing has many advantages: the same script can be reused for many data combinations, the script does not need to be changed every time new data is added, more test scenarios (valid, invalid, boundary values) can be covered easily, and non-technical people can also update test data without touching the code. Libraries such as Apache POI (for Excel files) and OpenCSV (for CSV files) are commonly used with Selenium to read external test data.

Common Types of Test Data:

Type of Test Data	Example	Purpose
Valid Data	Username: testuser, Password: Test@123	To verify the system works correctly with correct input
Invalid Data	Username: testuser, Password: wrong123	To verify proper error handling for wrong input
Boundary Data	Password of exactly 8 or 16 characters (min/max limit)	To verify system behaviour at the edge limits of input
Null / Blank Data	Username or Password field left empty	To verify mandatory field validation messages
Special Character Data	Password: Test@#$%123	To verify system accepts/rejects special characters correctly

Create Sticker
Reply to Sweet Bro...

Tools / Components / Instruments Required

1. Computer system with Windows/Linux Operating System
2. Java Development Kit (JDK) and Eclipse IDE
3. Selenium WebDriver library
4. Apache POI library (for reading/writing Excel .xlsx files) or OpenCSV library (for .csv files)
5. MS Excel (to create the test data file, e.g., TestData.xlsx)
6. Browser Driver (ChromeDriver) and Web Browser (Google Chrome)
7. TestNG framework (optional, useful with @DataProvider for data-driven tests)
8. A sample/demo Login web page to test

Procedure

Follow the steps given below to prepare and use external Test Data with Selenium:

1. Identify the test case that needs to be executed multiple times with different data (e.g., Login test case).
2. Open MS Excel and create a new file named TestData.xlsx. Create column headers such as TC_ID, Username, Password, and Expected Result.
3. Enter multiple rows of test data covering valid data, invalid data, and blank/boundary data for thorough testing.
4. Save the Excel file inside the Selenium project folder for easy access.
5. Add the Apache POI jar files (or Maven dependency) to the Selenium project to enable reading of Excel files.
6. Write Java code to open the Excel file using FileInputStream and load it into a Workbook object using XSSFWorkbook.
7. Read the required Sheet using getSheetAt(), then loop through each Row and read cell values (Username, Password) using getCell() and getStringCellValue().
8. Pass each row's data into the Selenium script so that the Login test case runs automatically for every row of data using a loop or a TestNG @DataProvider.
9. Compare the actual result with the Expected Result column for each row and note the Pass/Fail status.
10. Close the Workbook and FileInputStream after reading is complete, and document the final results.

Create Sticker
Reply to Sweet Bro...

6:25

Sent

Practical Diagram: Data-Driven Testing Process

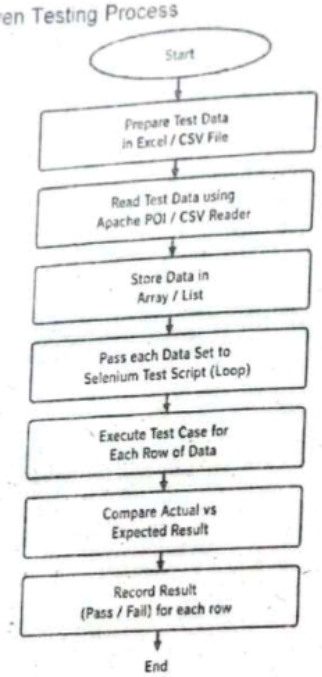


Fig 3.1: Data - Driven Testing Process Using Selenium

Practical Output: Sample Test Data prepared in Excel (Dummy Screenshot)

The following Excel sheet shows sample test data prepared for testing the Login module with multiple combinations of valid and invalid data:

TestData.xlsx - Excel

	A	B	C	D	E
1	TC_ID	Username	Password	Expected Result	Run
2	TC_01	testuser	Test@123	Login Success	Y
3	TC_02	testuser	wrong123	Invalid Password Msg	Y
4	TC_03		Test@123	Username Required Msg	Y
5	TC_04	baduser	Test@123	Invalid User Msg	Y
6	TC_05	testuser		Password Required Msg	N
7					

Create Sticker

Reply to Sweet Bro...

Practical Output: Sample Code to Read Excel Data using Apache POI (Dummy Screenshot)

The Selenium script below reads the above Excel file row by row and uses the values to perform login for each data set.

ReadExcelData.java - Selenium Project

```
1  FileInputStream fis = new FileInputStream("TestData.xlsx");
2  Workbook wb = new XSSFWorkbook(fis);
3  Sheet sheet = wb.getSheetAt(0);
4
5  for (int i = 1; i <= sheet.getLastRowNum(); i++) {
6      Row row = sheet.getRow(i);
7      String username = row.getCell(1).getStringCellValue();
8      String password = row.getCell(2).getStringCellValue();
9
10     driver.findElement(By.id("username")).sendKeys(username);
11     driver.findElement(By.id("password")).sendKeys(password);
12     driver.findElement(By.id("loginBtn")).click();
13
14     // Compare actual vs expected result here
15     driver.navigate().refresh();
16 }
17
18 wb.close();
19 fis.close();
```

Precautions

- Keep the column headers in the Excel/CSV file consistent and do not change their order once the script is written.
- Always close the Workbook and FileInputStream objects after reading data to avoid file lock or memory issues.
- Save the test data file in a proper location and use its correct relative/absolute path in the script.
- Avoid leaving unnecessary blank rows or extra spaces in the data file, as they may cause errors while reading.
- Keep a backup copy of the test data file before making changes, especially for large test suites.
- Do not hard-code test data in the script when following a data-driven approach; always fetch it from the external file.

Create Sticker
Reply to Sweet Bro...

6:25

Sent

4 Decision Table Preparation with Selenium

Introduction

Decision Table Testing is a black-box test design technique used when a software feature depends on a combination of multiple conditions, and each combination may lead to a different action or outcome. It is especially useful for testing business rules and logical conditions where simple positive/negative testing is not enough.

A Decision Table lists all possible combinations of input conditions (usually as Yes/No or True/False) in columns called 'Rules', along with the corresponding action or expected result for each rule. This technique ensures that no combination of conditions is missed while designing test cases, making testing more systematic and complete.

For example, in a Login page there are two main conditions: whether the Username is valid and whether the Password is valid. Since each condition can be True or False, there are 2 x 2 = 4 possible combinations (rules). Each of these rules is then converted into a separate test case, which can further be automated using Selenium WebDriver to save time and ensure accurate, repeatable execution.

Tools / Components / Instruments Required

1. Computer system with Windows/Linux Operating System
2. Java Development Kit (JDK) and Eclipse IDE (or any preferred IDE)
3. Selenium WebDriver library
4. Browser Driver (ChromeDriver) and Web Browser (Google Chrome)
5. MS Word / MS Excel (for preparing and documenting the Decision Table)
6. A sample/demo Login web page with Username, Password fields and a message area for errors
7. TestNG framework (optional, for organizing multiple test cases derived from the table)

Procedure

Follow the steps given below to prepare a Decision Table and automate it using Selenium:

1. Select the functionality to be tested and identify all the conditions that affect its behaviour

Create Sticker

Reply to Sweet Bro...

(e.g., for Login: Username Valid, Password Valid)

2. Determine the possible values for each condition. In most cases, conditions are binary (Yes/No or True/False).
3. Calculate the total number of rule combinations using the formula 2^n, where n is the number of conditions (for 2 conditions, 2^2 = 4 rules).
4. List every possible combination of the conditions as separate columns (rules) in a table.
5. For each rule/combination, identify and write the correct action or expected outcome of the system (e.g., Login Successful, Invalid Password error, etc.).
6. Prepare the final Decision Table with Conditions and Actions as rows, and Rules (combinations) as columns, marking 'X' for the action that applies to each rule.
7. Convert each rule of the Decision Table into an individual Test Case with proper Test Data, based on the condition values (Y/N) of that rule.
8. Write a Selenium WebDriver script that loops through the test data of all rules, enters the values into the Username and Password fields, clicks the Login button, and captures the resulting message.
9. Compare the actual message displayed by the application with the expected outcome defined in the Decision Table for that rule.
10. Record the Pass/Fail status for each rule after execution and document the final results.

Practical Output: Decision Table prepared for Login Functionality

	Rule 1	Rule 2	Rule 3	Rule 4
Conditions				
Username is Valid	Y	Y	N	N
Password is Valid	Y	N	Y	N
Actions				
Login Successful	X			
Show 'Invalid Password' Error		X		
Show 'Invalid Username' Error			X	X

Fig 4.1: Decision Table for ogin Functionality

Create Sticker
Reply to Sweet Bro...

Test Cases derived from the Decision Table:

Test Case ID	Derived From	Test Data (Input)	Expected Result
TC_01	Rule 1	Valid Username, Valid Password	Login Successful message displayed
TC_02	Rule 2	Valid Username, Invalid Password	'Invalid Password' error displayed
TC_03	Rule 3	Invalid Username, Valid Password	'Invalid Username' error displayed
TC_04	Rule 4	Invalid Username, Invalid Password	'Invalid Username' error displayed

Practical Diagram: Process of Decision Table Testing with Selenium

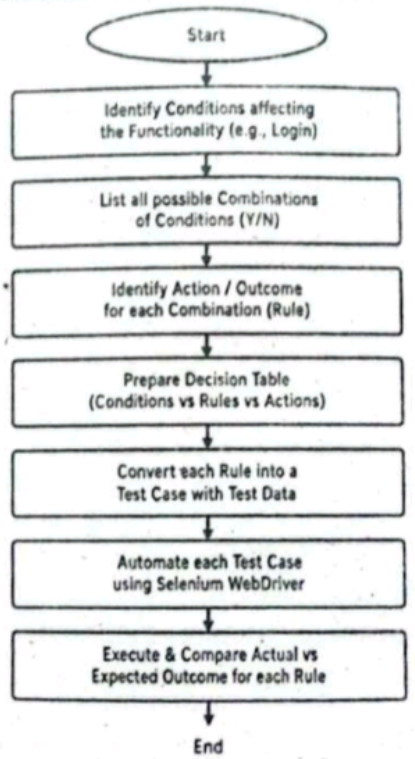


Fig 4.2: Process of Decision Table Testing using Selenium

Practical Output: Sample Selenium Script to Automate Decision Table Test Cases (Dummy Screenshot)

Create Sticker
Reply to Sweet Bro...

5 Behaviour-Driven Development (BDD) Examples Practice

Introduction

Behaviour-Driven Development (BDD) is a software development and testing approach that encourages collaboration between developers, testers, and non-technical business stakeholders (like clients or product owners). Instead of writing test cases in technical language, BDD describes the expected behaviour of the application in simple, plain English so that everyone in the team can understand it.

BDD scenarios are written using a special, easy-to-read language called Gherkin, using the keywords Given, When, and Then. 'Given' describes the starting condition, 'When' describes the action performed by the user, and 'Then' describes the expected result. These scenarios are saved in files with a .feature extension.

Cucumber is the most popular tool used to implement BDD. It reads the .feature file and matches each Gherkin step (Given/When/Then) with a corresponding Java method, called a Step Definition, which contains the actual Selenium WebDriver code to perform that action in the browser. This way, BDD combines readable documentation with automated testing in a single framework.

Common Gherkin Keywords used in BDD:

Keyword	Purpose	Example
Feature	Describes the overall functionality being tested	Feature: Login functionality of the website
Scenario	Describes one specific test case/behaviour	Scenario: Successful login with valid credentials
Given	Describes the initial context/precondition	Given user is on the Login page
When	Describes the action performed by the user	When user enters username as "testuser"
Then	Describes the expected outcome/result	Then user should be navigated to Home page
And / But	Used to add more steps of the same type	And user clicks on the Login button

Create Sticker
Reply to Sweet Bro...

Tools / Components / Instruments Required

1. Computer system with Windows/Linux Operating System
2. Java Development Kit (JDK) and Eclipse IDE (or IntelliJ IDEA)
3. Cucumber plugin/dependency for Eclipse (Cucumber-Java, Cucumber-JUnit)
4. Selenium WebDriver library
5. Maven (for managing project dependencies) or manually added jar files
6. Browser Driver (ChromeDriver) and Web Browser (Google Chrome)
7. A sample/demo Login web page to automate
8. JUnit framework (used along with Cucumber to run the Test Runner class)

Procedure

Follow the steps given below to write and execute a BDD example using Cucumber and Selenium:

1. Install the Cucumber plugin/dependency in Eclipse and create a new Maven project.
2. Add the required dependencies (cucumber-java, cucumber-junit, selenium-java) in the project's pom.xml file.
3. Create a new folder for feature files and create a file named Login.feature inside it.
4. Write the Feature title and one or more Scenarios inside the feature file using Given, When, Then, and And keywords in plain English, describing the Login behaviour.
5. Create a Step Definition Java class (e.g., LoginSteps.java) and generate/write methods corresponding to each Gherkin step using annotations such as @Given, @When, and @Then.
6. Inside each step definition method, write the actual Selenium WebDriver code (e.g., driver.get(), findElement(), sendKeys(), click()) to perform the real action on the browser.
7. Create a Test Runner class using @RunWith(Cucumber.class) and @CucumberOptions, specifying the path of the feature file and the step definitions package (glue path).
8. Run the Test Runner class as a JUnit test; Cucumber will automatically match each Gherkin step with its step definition and execute the Selenium code.
9. Observe the console output to check whether each step Passed or Failed, and generate an HTML report (if configured) for a detailed view of the results.

Practical Diagram: BDD Process Flow using Cucumber and Selenium

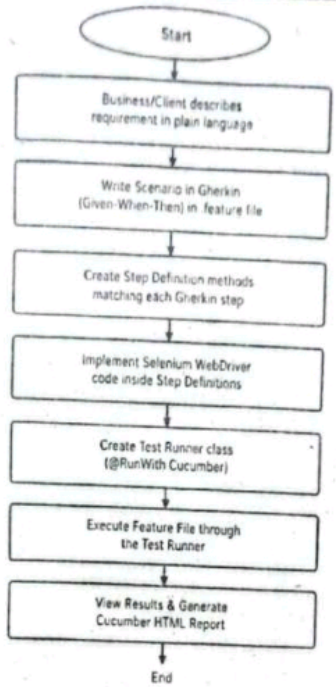


Fig 5.1: BDD Process Flow using Cocumber and Selenium

Practical Output: Sample Feature File written in Gherkin (Dummy Screenshot)

The following feature file describes two scenarios for the Login functionality in plain, business-readable language:

Login.feature - Cucumber Feature File

```
1  Feature: Login functionality of the website
2
3  Scenario: Successful login with valid credentials
4      Given user is on the Login page
5      When user enters username as "testuser"
6      And user enters password as "Test@123"
7      And user clicks on the Login button
8      Then user should be navigated to Home page
9
10 Scenario: Unsuccessful login with invalid password
11     Given user is on the Login page
12     When user enters username as "testuser"
13     And user enters password as "wrong123"
14     And user clicks on the Login button
15     Then an "Invalid Password" error should be displayed
```

Create Sticker
Reply to Sweet Bro...